Projektuppgift

Frontend-baserad webbutveckling

Projekt VanTrail
En vanlife-applikation

Fredrika Stjernlöf

Mittuniversitetet
MID SWEDEN UNIVERSITY

Campus Härnösand Universitetsbacken 1, SE-871 88. **Campus Sundsvall** Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 50 00, Fax: +46 (0)771 97 50 01.

MITTUNIVERSITETET
Avdelningen för informationssystem och -teknologi

Författare: Fredrika Stjernlöf, frst2501@student.miun.se
Utbildningsprogram: Webbutveckling, 120 hp
Huvudområde: Datateknik
Termin, år: VT, 2026

Sammanfattning

Målet med detta projekt har varit att utveckla en webbapplikation som underlättar planering av roadtrips och vanlife-resor genom att kombinera ruttplanering, stopp längs vägen och väderinformation i ett och samma gränssnitt. Applikationen VanTrail är en frontend-baserad lösning utan ramverk, byggd med JavaScript, Vite och externa API:er.

Arbetet har följt en iterativ utvecklingsprocess där initiala idéer och mockups successivt har omarbetats utifrån tekniska begränsningar och användbarhet. Applikationen använder OpenRouteService för rutter, Overpass API för att hämta sevärdheter och Open-Meteo för väderdata. Leaflet används för kartvisualisering.

Under utvecklingen uppstod flera utmaningar, särskilt kring CORS-problem och begränsningar i att använda externa API:er direkt från frontend. Detta påverkade bland annat möjligheten att använda reverse geocoding, vilket i sin tur ledde till förändringar i funktionaliteten.

Resultatet är en fungerande webbapplikation där användaren kan planera en rutt, se relevanta stopp längs vägen, filtrera dessa samt få förslag på platser med bra väder. Applikationen är publicerad och tillgänglig online.

Projektet visar både möjligheterna och begränsningarna med att bygga en komplett applikation enbart med frontend och öppna API:er.

Innehållsförteckning

Sammanfattning **2**

Innehållsförteckning **3**

1 Introduktion / Inledning **4**
1.1 Bakgrund och problemmotivering 4
1.2 Övergripande syfte 4
1.3 Avgränsningar 5
1.4 Konkreta och verifierbara mål 5
1.5 Författarens bidrag 6

2 Teori **7**

3 Metod **9**

4 Konstruktion **11**

5 Resultat **13**

6 Slutsatser **14**

Källförteckning **16**

Bilaga A: Målgruppsanalys **18**

Bilaga B: Mockups **20**

Introduktion / Inledning

1.1 Bakgrund och problemmotivering

Intresset för roadtrips och vanlife har ökat kraftigt de senaste åren. Samtidigt finns det få digitala verktyg som samlar ruttplanering, stopp längs vägen och väderinformation i ett och samma gränssnitt. I praktiken innebär det ofta att användaren behöver växla mellan flera olika tjänster för att planera en resa, vilket gör processen både mer tidskrävande och mindre överskådlig.

En central del av idén bakom projektet uppstod ur ett konkret behov: att enkelt kunna hitta platser med bra väder utan att behöva söka manuellt på flera olika orter. Befintliga vädertjänster visar visserligen prognoser för enskilda platser, men saknar ofta möjlighet att ge en överblick över *var* förhållandena är som mest gynnsamma inom ett större område.

Utifrån detta behov utvecklades idén vidare till en mer komplett lösning. Genom att kombinera ruttplanering med information om intressanta stopp, såsom platser för mat, toaletter, utsiktsplatser och övernattning, skapas en applikation som inte bara visar vägen utan även stödjer själva upplevelsen längs resan.

Projektet är en så kallad mashup-applikation, där flera externa API:er kombineras för att skapa en samlad funktionalitet. Detta har påverkat både design och tekniska val i utvecklingen.

1.2 Övergripande syfte

Projektets syfte har varit att utveckla en webbapplikation som kombinerar:

- ruttplanering
- relevanta stopp längs vägen
- väderinformation

Målet var att skapa ett verktyg som ger användaren bättre beslutsunderlag vid planering av resor. Genom att samla flera typer av information i ett gemensamt gränssnitt blir det enklare att få en helhetsbild av resan och fatta mer informerade beslut, exempelvis kring vägval, stopp och destinationer.

1.3 Avgränsningar

Projektet är avgränsat till:

- en frontend-baserad lösning utan egen backend
- användning av öppna och kostnadsfria API:er

Valet av kostnadsfria API:er innebär vissa begränsningar i funktionalitet och datatillgång, vilket har påverkat vilka funktioner som kunnat implementeras och hur de fungerar i praktiken.

Applikationen inkluderar inte användarhantering eller permanent datalagring, utan använder enbart webbläsarens localStorage för enklare lagring.

1.4 Konkreta och verifierbara mål

För att uppfylla projektets syfte har följande konkreta mål definierats:

- Användaren ska kunna ange en start- och slutpunkt för att generera en rutt
- Applikationen ska kunna hämta och visa relevanta stopp längs rutten baserat på geografiska data.
- Det ska vara möjligt att filtrera stopp utifrån olika kategorier, exempelvis mat, toaletter, utsiktsplatser och övernattning
- Användaren ska kunna ta del av väderinformation kopplad till platser längs rutten eller inom ett valt område
- Informationen ska presenteras visuellt i ett kartbaserat gränssnitt

Målen är utformade så att de är möjliga att verifiera genom att funktionerna implementeras och kan testas i den färdiga applikationen.

1.5 Författarens bidrag

Arbetet är genomfört individuellt. Samtliga delar – idé, design, implementation, testning och dokumentation – har utförts av författaren.

2 Teori

Applikationen bygger på flera externa API:er och webbaserade teknologier som tillsammans möjliggör funktionaliteten.

OpenRouteService används för att generera rutter mellan två koordinater. Tjänsten erbjuder även geocoding, vilket innebär att adresser kan omvandlas till koordinater [1].

Open-Meteo används för att hämta väderdata. API:et erbjuder både aktuell väderinformation och prognoser utan krav på API-nyckel [2].

Overpass API används för att hämta data från OpenStreetMap. Genom att formulera queries kan specifika typer av objekt, som restauranger, campingplatser och utsiktsplatser, hämtas inom geografiska områden [3].

Leaflet används som kartbibliotek för att visualisera rutter och markörer [4].

Applikationen använder även Web APIs som:

- Fetch API för HTTP-anrop [5]
- Geolocation API för användarens position [6]
- localStorage för att spara tema [9]

En viktig begränsning i projektet är CORS (Cross-Origin Resource Sharing), vilket styr vilka resurser en webbsida får hämta från andra domäner. Detta påverkade särskilt användningen av vissa API-anrop [7].

Progressive Web Apps (PWA) är webbapplikationer som byggs med webbtekniker men kan ge en mer app-liknande upplevelse, bland annat genom att kunna installeras direkt från webbläsaren. I detta projekt har applikationen konfigurerats som en PWA, vilket möjliggör installation och användning utan att behöva installeras via en appbutik [12].

Tillgänglighet är också relevant i projektet. WCAG är en uppsättning riktlinjer för att göra webbinnehåll mer tillgängligt för personer med funktionsnedsättning. I applikationen har detta beaktats genom exempelvis användning av skip-links och semantiska attribut i gränssnittet [13].

3 Metod

Projektet har följt en iterativ arbetsprocess där applikationen utvecklades stegvis och testades löpande under arbetets gång. Arbetet inleddes med en idé, identifiering av målgrupp och tidiga skisser, som sedan låg till grund för val av funktioner och tekniska lösningar.

Utvecklingen genomfördes modulärt, där olika delar av applikationen delades upp i separata JavaScript-filer för exempelvis rutter, stopp, väder, karta, gränssnitt och tema. På så sätt blev det möjligt att bygga och testa funktionerna var för sig innan de kopplades samman i applikationens huvudflöde.

Arbetet började med grundstruktur, layout och initiering av kartfunktionalitet. Därefter implementerades ruttplaneringen, där användarens valda platser först omvandlas till koordinater och sedan används för att hämta en rutt. När rutten fungerade byggdes funktionalitet för stopp längs vägen, där data hämtas från OpenStreetMap via Overpass API, normaliseras, sorteras längs rutten och filtreras efter valda kategorier innan de presenteras i gränssnittet.

Efter detta utvecklades väderfunktionen. Där används användarens position eller en vald startpunkt som utgångspunkt för att generera kandidatplatser inom ett visst område. För dessa platser hämtas väderdata, som sedan bearbetas, grupperas och sorteras för att lyfta fram platser med bättre väderförhållanden. För att göra funktionen mer tålig används *Promise.allSettled(),* vilket innebär att resultat kan visas även om vissa enskilda API-anrop misslyckas [8].

Som en mindre kompletterande funktion implementerades även stöd för temaval, där användarens val sparas i webbläsarens *localStorage* och återanvänds vid nästa besök [9]. Detta bidrog till en mer sammanhållen användarupplevelse.

Under arbetets gång justerades funktioner utifrån vad som var tekniskt möjligt att genomföra med en frontend-baserad lösning och de API:er som användes. Arbetssättet var därför iterativt även i praktiken: en första version av en funktion byggdes, testades i webbläsaren och förbättrades därefter stegvis.

Testning har främst skett genom kontinuerlig manuell testning i webbläsare, där varje delmoment verifierades innan nästa funktion utvecklades vidare. På så sätt kunde fel upptäckas tidigt och funktionerna successivt integreras i den färdiga applikationen. Utöver manuell testning har koden även validerats med hjälp av HTML- och CSS-valideringsverktyg för att säkerställa korrekt struktur och syntax [10] [11].

4 Konstruktion

Applikationen är uppbyggd modulärt med separata JavaScript-filer för olika funktioner. Detta gör det möjligt att dela upp logiken och hantera varje del av systemet separat.

De mest centrala modulerna är:

- route.js – hanterar ruttberäkning
- poi.js – hanterar stopp längs rutten
- weather.js – hanterar väderdata
- map.js – ansvarar för kartvisualisering
- ui.js – hanterar användargränssnittet

Rutter hämtas från OpenRouteService och ritas ut på kartan med hjälp av Leaflet. För att visa relevanta stopp längs rutten används Overpass API, där queries genereras baserat på ruttens koordinater. För att begränsa datamängden används sampling av ruttens koordinater.

Väderfunktionen bygger på att generera kandidatplatser inom ett visst område kring en startpunkt. För dessa platser hämtas väderdata, som sedan används för att identifiera områden med mer gynnsamma väderförhållanden.

Applikationen är uppbyggd med HTML, där strukturen är uppdelad på flera sidor med gemensamma komponenter som header och navigation. Tillgänglighet har beaktats genom exempelvis användning av skip-links och semantiska attribut i gränssnittet.

Stylingen är byggd med SCSS och uppdelad i flera delar, såsom variabler, teman, layout och komponenter. Detta gör det enklare att hålla koden struktu-

rerad och återanvändbar. Temasystemet är kopplat till dessa stilar och möjliggör byte mellan olika visuella uttryck, där användarens val sparas i webbläsarens localStorage.

Applikationen innehåller även funktioner för filtrering av stopp, visning av detaljer i modaler samt ett temasystem som påverkar både gränssnitt och visuellt uttryck.

Projektet använder även en automatiserad arbetsprocess för att bygga och optimera resurser, inklusive kompilering av SCSS och optimering av bilder.

Applikationen är konfigurerad som en Progressive Web App (PWA), vilket möjliggör installation och användning direkt från webbläsaren.

5 Resultat

Resultatet av projektet är en fungerande webbapplikation där användaren kan planera och utforska resor i ett samlat gränssnitt.

Applikationen gör det möjligt att:

- ange start och destination
- generera och visualisera en rutt på karta
- visa relevanta stopp längs vägen
- filtrera stopp utifrån olika kategorier
- interagera med stopp och visa dem på kartan
- identifiera platser med gynnsamma väderförhållanden
- byta tema utifrån användarens preferenser

Funktionerna är integrerade i ett gemensamt gränssnitt där karta, lista och filtrering samverkar för att ge en översikt av resan.

Applikationen är responsiv och anpassad för olika skärmstorlekar samt fungerar i moderna webbläsare.

6 Slutsatser

Projektet uppnådde sina huvudsakliga mål, men inte utan utmaningar. Resultatet är en fungerande webbapplikation där de centrala funktionerna har implementerats, även om vissa delar fick anpassas under arbetets gång.

En av de största insikterna var omfattningen av projektet. Det som initialt upplevdes som en tydlig och hanterbar idé visade sig vara mer komplext i praktiken. Trots en planerad struktur och uppdelning av kod i moduler växte projektet snabbt, vilket gjorde det svårt att överblicka helheten under utvecklingen.

En central utmaning var begränsningarna med att arbeta enbart i frontend. Problem kopplade till CORS och externa API:er påverkade särskilt möjligheten att implementera reverse geocoding. Tanken var att alla stopp skulle kompletteras med närmaste ort eller kommun, men detta visade sig inte vara möjligt att genomföra fullt ut. Flera försök gjordes för att lösa problemet, men till slut fick funktionaliteten förenklas för att kunna färdigställa applikationen inom tidsramen.

Ett konkret exempel på detta är väderfunktionen, där den ursprungliga idén var att visa tydligt “soliga platser”. När funktionaliteten vidareutvecklades för att ge fler träffar, exempelvis genom att inkludera uppehållsväder och lätt molnighet, uppstod nya tekniska begränsningar som påverkade möjligheten att presentera platserna på det sätt som först var tänkt. Detta ledde till att presentationen fick förenklas.

Ett annat exempel på detta var användningen av Overpass API, där en alternativ publik endpoint fick användas som fallback för att göra funktionen mer robust vid belastning eller tillgänglighetsproblem [14].

Projektet visar tydligt att det ofta krävs kompromisser mellan idé och teknisk verklighet, särskilt vid användning av externa API:er och en frontend-

baserad lösning. Samtidigt bidrog dessa begränsningar till en djupare förståelse för hur webben fungerar i praktiken.

En annan viktig lärdom är vikten av iteration. Flera delar av applikationen utvecklades i flera steg, där funktioner först implementerades i en enklare form och därefter förbättrades. Detta arbetssätt visade sig vara nödvändigt för att komma framåt i projektet, även om det ibland innebar att tidigare lösningar behövde omarbetas.

Sammanfattningsvis har projektet gett en ökad förståelse för både tekniska begränsningar och arbetsprocesser inom webbutveckling. Trots vissa förenklingar i funktionaliteten uppfyller applikationen sina kärnmål och fungerar som ett sammanhållet verktyg för planering av resor.

Källförteckning

[1] OpenRouteService – Geocoding: https://openrouteservice.org/dev/#/api-docs/geocode
Publicerad: ej angivet. Hämtad 2026-03-21

[2] Open-Meteo – Weather Forecast API: https://open-meteo.com/en/docs
Publicerad: ej angivet. Hämtad: 2026-03-21

[3] OpenStreetMap Wiki - Overpass API: https://wiki.openstreetmap.org/wiki/Overpass_API
Senast uppdaterad: 2026-03-22. Hämtad: 2026-03-21

[4] Leaflet – Översikt: https://leafletjs.com/index.html
Publicerad: ej angivet. Hämtad 2026-03-21

[5] MDN – Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
Senast uppdaterad: 2026-01-08. Hämtad: 2026-03-21

[6] MDN – Geolocation API: https://developer.mozilla.org/en-US/docs/Web/API/Geolocation_API
Senast uppdaterad: 2026-03-10. Hämtad: 2026-03-21

[7] MDN – CORS https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS
Senast uppdaterad: 2025-11-30. Hämtad: 2026-03-21

[8] MDN – Promise.allSettled(): https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/allSettled
Senast uppdaterad: 2025-07-10. Hämtad: 2026-03-23

[9] MDN – localStorage property: https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage
Senast uppdaterad: 2025-11-30. Hämtad: 2026-03-23

[10] HTML-validator: https://validator.w3.org
Hämtad: 2026-03-23

[11] CSS-validator: https://jigsaw.w3.org/css-validator/
Hämtad: 2026-03-23

[12] Progressive web apps: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps
Senast uppdaterad: 2025-08-25. Hämtad: 2026-03-23

[13] W3C – WCAG 2 Overview https://www.w3.org/WAI/standards-guidelines/wcag
Publicerad: 2005-07-XX. Hämtad: 2026-03-23

[14] OpenStreetMap Wiki – Public Overpass API instances: https://wiki.openstreetmap.org/wiki/Overpass_API#Public_Overpass_API_instances
Senast uppdaterad: 2026-03-22. Hämtad: 2026-03-23

Bilaga A

Målgruppsanalys för VanTrail

Övergripande målgrupp

Applikationen riktar sig till personer som reser med bil, husbil eller van och som vill planera sina resor på ett flexibelt sätt. Användarna har behov av att snabbt kunna få en överblick över rutter, stopp och väderförhållanden utan att behöva använda flera olika tjänster.

Primära målgrupper

1. Vanlife-resenärer och roadtrip-resenärer

- Reser ofta och fattar beslut längs vägen
- Behöver hitta stopp för mat, vila och övernattning
- Vill kunna anpassa resan efter väder

Digitala behov:

- Snabb ruttplanering
- Tydlig kartvy
- Filtrering av stopp
- Översikt över väder

2. Planerande resenärer

- Vill få överblick över resan i förväg
- Jämför rutter och stopp

Digitala behov:

- Kombination av karta och lista
- Enkel och tydlig struktur

Användarkrav

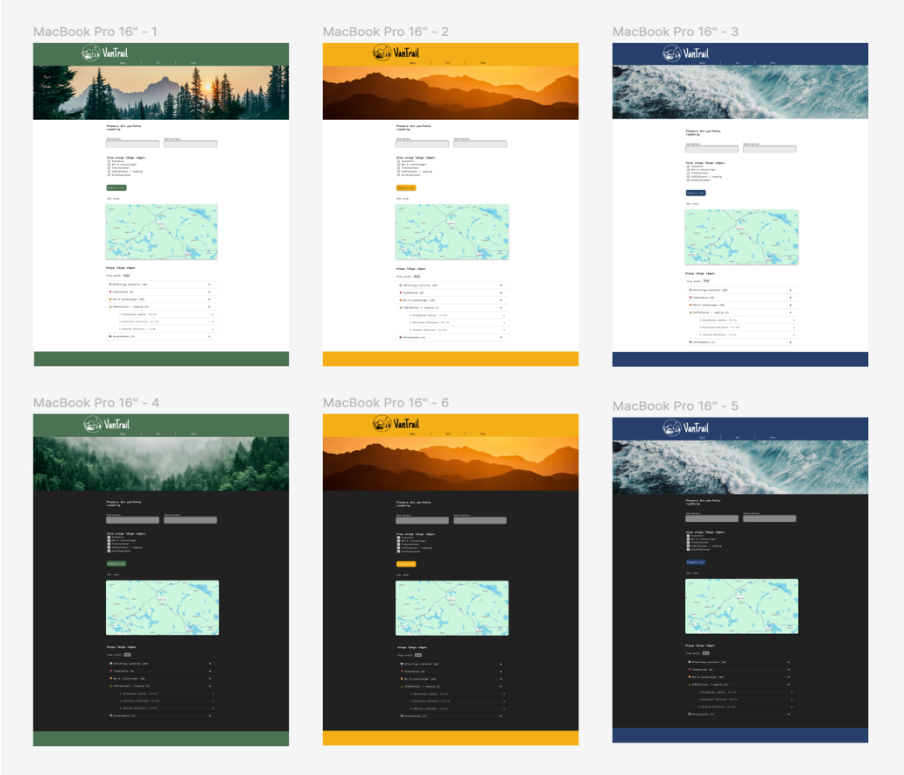
- Enkel navigering
- Snabb åtkomst till funktioner
- Tydlig koppling mellan karta och information
- Responsiv design för mobil och dator
- Möjlighet att kombinera rutt, stopp och väder i ett gränssnitt

Sammanfattning

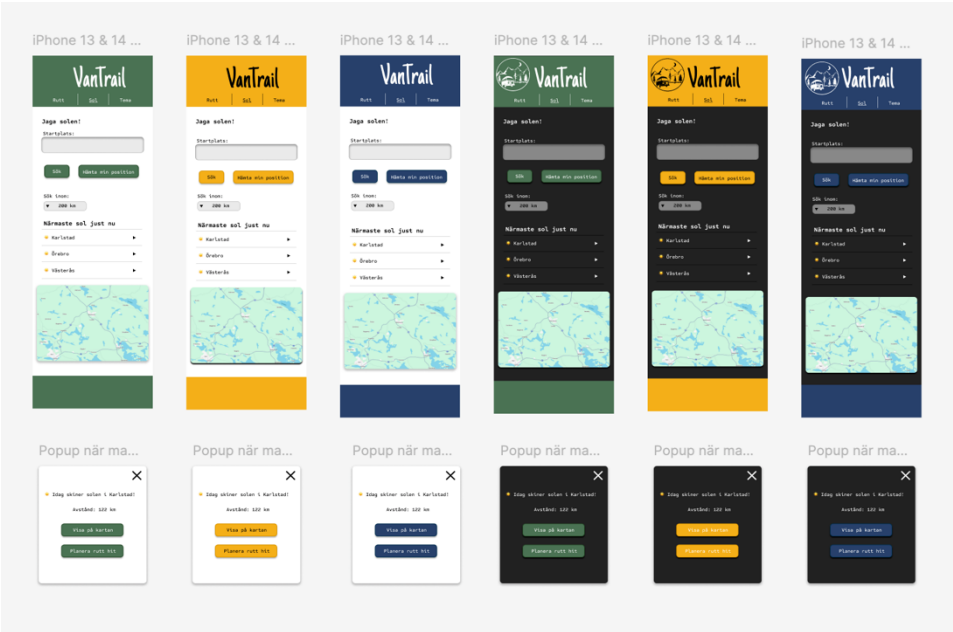
Applikationen är utformad för att stödja ett flexibelt och visuellt sätt att planera resor. Genom att samla flera funktioner i ett gränssnitt kan användaren snabbt få en överblick och fatta beslut under resans gång.

Bilaga B

Mockups



Figur 1 – Mockup för startsida desktop, ljust och mörkt läge



Figur 2 – Mockup för sök-solsidan mobil, ljust och mörkt läge